

Приватний заклад вищої освіти
Одеський технологічний університет «ШАГ»
Кафедра інформаційних технологій та фундаментальної підготовки

випускна кваліфікаційна робота бакалавра

**Інформаційна платформа розповсюдження мультимедійного контенту за
моделлю підписки (на прикладі сервісу Bazinga)**

на здобуття бакалаврського рівня вищої освіти
зі спеціальності «F3 Комп'ютерні науки»

Виконавці проекту:

№	П.І.Б.	Ролі	Група
1	Маймескул Єгор Олегович	Full-Stack	КНП-221
2	Руденко Олена Костянтинівна	Full-Stack	КНП-221

Автор звіту: Руденко Олена Костянтинівна

Керівник	Доцент кафедри ІТ та ФП, к.т.н., [Прізвище І. П. керівника]
----------	---

Одеса – 2025

АНОТАЦІЯ

УДК 004.9

Руденко О. К. Інформаційна платформа розповсюдження мультимедійного контенту за моделлю підписки (на прикладі сервісу Bazinga). Підсистема каталогу контенту, агрегатора метаданих та потокового відео. — Пояснювальна записка до випускної кваліфікаційної роботи на здобуття бакалаврського рівня вищої освіти зі спеціальності «ФЗ Комп'ютерні науки» (0613 Software and applications development and analysis). — Одеса: ОТУШ, 2025.

Об'єктом дослідження є процеси агрегування, уніфікованого подання та розповсюдження метаданих мультимедійного контенту (коміксів, манги, аніме та серіалів) з відкритих джерел у складі інформаційної системи за моделлю підписки.

Метою роботи є підвищення відмовостійкості та швидкодії програмного інтерфейсу прикладного рівня (API) до агрегатора метаданих, що уніфікує різноманітні зовнішні джерела (Jikan v4, akabab/superhero-api, TVMaze, Open Library) в єдиній моделі контенту і функціонує в умовах обмежень темпу звернень та можливих відмов провайдерів. Уніфіковане візуальне подання карток в інтерфейсі користувача є наслідком цієї уніфікації на рівні моделі та API і не виступає самостійним предметом дослідження.

Методи дослідження утворені суперпозицією методів системного аналізу, об'єктно-орієнтованого моделювання, порівняльного аналізу джерел даних і проєктних рішень та експериментального випробування. Інструментальне забезпечення: платформа .NET 9 (ASP.NET Core 9) з кешуванням у пам'яті (IMemoryCache); клієнтський застосунок на React 18 і TypeScript 5; відкриті джерела метаданих без авторизації — Jikan v4 (MyAnimeList), akabab/superhero-api, TVMaze, Open Library.

Результати та їх новизна. Спроектовано та впроваджено кешувальний агрегатор метаданих, що уніфікує різноманітні відкриті джерела в єдиній інформаційній моделі контенту, а також підсистему персональних колекцій з прив'язкою до профілю та потокового перегляду відео. Новизна полягає: у синтезі уніфікованої моделі подання різноманітного контенту (єдиний інтерфейсний елемент картки для коміксів, манги, аніме та серіалів); у застосуванні стратегії «кеш-обабіч» (Cache-Aside) з диференційованими інтервалами актуальності та обмеженням темпу звернень для обходу лімітів зовнішніх джерел; у відв'язуванні клієнта від конкретного постачальника метаданих через комунікаційні інтерфейси, що дозволило замінити припинений комерційний сервіс відкритим джерелом без зміни клієнтської частини.

Основні характеристики. Реалізовано єдину точку доступу до метаданих (18 ендпоінтів); статичний набір даних персонажів обсягом близько 1 МБ завантажується одноразово й обробляється у пам'яті; повторні звернення до каталогу обслуговуються з кешу за час порядку мілісекунд.

Сфера застосування — інформаційні системи розповсюдження цифрового контенту, агрегатори метаданих, системи е-комерції контенту. *Значущість роботи* визначається відповідністю наряду державної політики щодо розвитку цифрової економіки та інформаційного суспільства. *Взаємозв'язок з іншими роботами* — підсистема є складовою командного проєкту Bazinga; підсистема облікових записів, профілів та підписок реалізована першим виконавцем.

Пояснювальна записка складається зі вступу, технічного завдання, трьох розділів, висновків, переліку використаних джерел та додатка з прикладами програмного коду. При складанні звіту використано 19 джерел інформації.

ЗМІСТ

- ЗМІСТ
 - ВСТУП
 - ТЕХНІЧНЕ ЗАВДАННЯ ДО ПРОЄКТУ
 - РОЗДІЛ 1. ЗАГАЛЬНА ХАРАКТЕРИСТИКА СИСТЕМИ ТА ПОРІВНЯЛЬНИЙ АНАЛІЗ
 - РОЗДІЛ 2. ПРОЄКТУВАННЯ ТА РЕАЛІЗАЦІЯ ПІДСИСТЕМИ
 - РОЗДІЛ 3. ВИПРОБУВАННЯ ТА ОЦІНКА РЕЗУЛЬТАТІВ
 - ВИСНОВКИ
 - ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ
 - ДОДАТОК А. ПРИКЛАДИ ПРОГРАМНОГО КОДУ
-

ВСТУП

Оцінка сучасного стану об'єкта дослідження. Сучасні інформаційні системи розповсюдження мультимедійного контенту дедалі частіше будуються як агрегатори, що поєднують контент і метадані з кількох джерел. Якість такого контентного шару визначає швидкість доступу до контенту, повноту його опису (обкладинки, описи, жанри) та зручність пошуку. Водночас побудова власної редакційної бази метаданих на десятки тисяч одиниць контенту є ресурсомісткою, тому поширеною є інтеграція із зовнішніми джерелами метаданих.

Світові тенденції розв'язання задачі. Провідні системи застосовують уніфіковане подання різноманітного контенту, наскрізний пошук за кількома категоріями одночасно, персональні колекції на рівні профілю та потоковий перегляд без завантаження файлів. На рівні інженерних рішень типовими є кешування зовнішніх звернень для зниження навантаження й затримок, а також відв'язування системи від конкретного постачальника даних задля стійкості до змін у його доступності.

Актуальність роботи. Побудова відмовостійкого програмного інтерфейсу до агрегатора метаданих, здатного об'єднувати різноманітні зовнішні джерела й зберігати працездатність за їх часткової відмови чи обмежень темпу звернень, є передумовою функціонування сучасних інформаційних систем розповсюдження контенту, що й обумовлює актуальність роботи.

Умови невизначеності. Задача побудови агрегатора сформульована в умовах істотної неповної визначеності: зовнішні джерела метаданих мають різні формати відповідей, накладають обмеження на темп звернень, можуть бути тимчасово недоступними, а окремі з них — узагалі припинитися. Зокрема, у ході роботи комерційний сервіс метаданих коміксів (Marvel Developer API) став недоступним, що потребувало дослідницького переходу на альтернативне відкрите джерело без зміни клієнтської частини. Це підтверджує недетермінований характер задачі та потребу проєктування під невизначеність.

Наукова новизна. Новизна роботи полягає у синтезі уніфікованої інформаційної моделі подання різноманітного контенту та у застосуванні стратегії кешування з обмеженням темпу звернень для обходу лімітів зовнішніх джерел. Додатковим елементом новизни є архітектурне відв'язування клієнта від постачальника метаданих через комунікаційні інтерфейси, що забезпечує підмінність джерела без модифікації решти системи. Порівняльний аналіз (розділ 1) показує, що поєднання цих властивостей вирізняє запропоноване рішення серед систем-аналогів.

Мета і задачі. Метою роботи є проектування, реалізація та впровадження відмовостійкого програмного інтерфейсу прикладного рівня (API) до кешувального агрегатора метаданих інформаційної системи Bazinga, що уніфікує доступ до різномірних зовнішніх джерел та забезпечує стабільну роботу підсистеми в умовах обмежень темпу звернень і можливих відмов провайдерів. Для досягнення мети поставлено такі задачі:

1. провести огляд інформаційних систем розповсюдження контенту та порівняльний аналіз доступних зовнішніх джерел метаданих за програмними інтерфейсами доступу, встановити критерії вибору джерел;
2. синтезувати уніфіковану інформаційну модель контенту (DTO) та спроектувати кешувальний агрегатор за стратегією Cache-Aside із диференційованими інтервалами актуальності;
3. реалізувати серверні комунікаційні інтерфейси (контролер `MetadataController`) та сервіси інтеграції з джерелами метаданих за принципом інверсії залежностей; впровадити обмеження темпу звернень для дотримання лімітів зовнішніх API;
4. забезпечити підмінність постачальника метаданих через комунікаційний інтерфейс (на прикладі заміни припиненого Marvel Developer API на Open Library без зміни клієнтської частини);
5. реалізувати клієнтських споживачів API — наскрізний пошук за кількома джерелами, підсистему персональних колекцій з прив'язкою до профілю та сторінку потокового перегляду відео;
6. провести випробування API та підсистеми, зокрема в умовах відмови та обмежень зовнішніх джерел, та оцінити відповідність технічному завданню.

Сфера застосування та взаємозв'язок з іншими роботами. Результати застосовні в інформаційних системах розповсюдження контенту та агрегаторах метаданих. Робота є складовою командного проекту; підсистема облікових записів, профілів та платіжно-підписної моделі реалізована першим виконавцем і описана в окремій пояснювальній записці.

ТЕХНІЧНЕ ЗАВДАННЯ ДО ПРОЄКТУ

Загальний опис

Програмне забезпечення реалізує контентний шар розподіленої інформаційної системи Bazinga — кешувальний агрегатор метаданих, уніфікований каталог коміксів, манги, персонажів і серіалів, наскрізний пошук, потоковий перегляд відео та персональні колекції користувача. Підсистема є спільною основою двох предметних складових системи — Bazinga Comics (комікси й манга) та BazingaTV (потокове відео).

Призначення

Підсистема призначена для виконання таких користувацьких сценаріїв:

- перегляд каталогу коміксів, манги, персонажів та серіалів з обкладинками, описами й жанрами;
- наскрізний пошук одночасно за кількома типами контенту з відкриттям детального подання без переходу на іншу сторінку;
- збереження одиниць контенту до персональних колекцій («Бібліотека» для коміксів/манги та «Мій список» для відео) з прив'язкою до активного профілю;
- потоковий перегляд відеоконтенту у вбудованому програвачі;
- фільтрація та посторінкове гортання каталогу.

Функціональні вимоги до підсистеми

Каталог і метадані:

- єдина точка доступу до метаданих з кількох зовнішніх джерел;
- уніфіковане подання різноманітного контенту однаковими інтерфейсними елементами;
- фільтрація за жанрами/видавцями та посторінкове гортання;
- виключення контенту для дорослих із загальнодоступних добірок.

Пошук:

- наскрізний пошук одночасно за коміксами, мангою, аніме та персонажами;
- відкриття детального подання результату безпосередньо в накладенні (без переходу на іншу сторінку).

Персональні колекції:

- збереження одиниць контенту до колекцій «Бібліотека» та «Мій список»;
- прив'язка колекцій до активного профілю (кожен профіль має власні колекції);

- розподіл колекції за типами контенту (усе / комікси / манга та усе / аніме / серіали).

Потокове відео:

- головна сторінка BazingaTV з добірками контенту та героїчним блоком;
- повноекранний перегляд у вбудованому програвачі.

Нефункціональні вимоги

- кількість звернень до зовнішніх джерел мінімізується кешуванням; темп звернень не повинен перевищувати обмежень джерел;
- повторні звернення до каталогу обслуговуються з кешу за час порядку мілісекунд;
- за відмови зовнішнього джерела підсистема зберігає працездатність (порожня добірка чи резервний набір замість аварійного завершення);
- клієнтська частина адаптивна; підтримуються актуальні версії основних браузерів.

Архітектура та технології

Серверна частина — ASP.NET Core 9 з кешуванням у пам'яті та фабрикою HTTP-клієнтів; клієнтська — React 18 + TypeScript зі складанням засобами Vite. Зовнішні джерела метаданих: Jikan v4 (MyAnimeList) — аніме й манга; akabab/superhero-api — персонажі коміксів; TVMaze — серіали; Open Library — комікси. Розгортання — Docker і Docker Compose.

РОЗДІЛ 1. ЗАГАЛЬНА ХАРАКТЕРИСТИКА СИСТЕМИ ТА ПОРІВНЯЛЬНИЙ АНАЛІЗ

Інформаційна система Bazinga є розподіленою клієнт-серверною системою, побудованою за архітектурним стилем «односторінковий застосунок поверх комунікаційного протоколу REST». Цей розділ присвячено загальній характеристиці системи та порівняльному аналізу наявних рішень, за результатами якого обґрунтовано предмет розроблення контентної підсистеми.

Контентний шар, що є предметом цієї роботи, реалізує єдину точку доступу до метаданих, уніфікований каталог, наскрізний пошук, потоковий перегляд і персональні колекції. Серверна частина застосовує кешування у пам'яті та фабрику HTTP-клієнтів для звернень до зовнішніх джерел; клієнтська частина уніфікує подання різноманітного контенту спільними інтерфейсними компонентами.

Порівняльний аналіз наявних рішень. Для обґрунтування предмета розроблення проведено порівняння інформаційних систем-аналогів за ознаками, що стосуються виявлення та споживання контенту. Результати наведено у таблиці 1.1.

Таблиця 1.1 — Порівняльний аналіз засобів виявлення та споживання контенту

Ознака / Система	Netflix	Crunchyroll	ComiXology	Bazinga
Єдиний каталог відео, коміксів і манги	—	—	—	+
Наскрізний пошук одночасно за кількома типами контенту	—	—	—	+
Уніфіковане подання різноманітного контенту	—	—	—	+
Персональні колекції на рівні профілю	+	—	—	+
Незалежність від конкретного джерела метаданих	—	—	—	+
Відмовостійке агрегування з кешуванням	—	—	—	+

Як видно з таблиці 1.1, жоден рядок не містить ознаки одночасно в усіх системах, що підтверджує диференційність обраних критеріїв. Серед систем-аналогів персональні колекції на рівні профілю реалізовано лише в Netflix, який і взято за взірець щодо цієї властивості. Водночас жодна з наявних систем не поєднує єдиного каталогу різноманітного контенту з наскрізним пошуком, уніфікованим поданням та незалежністю від конкретного джерела метаданих — саме усунення цих «мінусів» становить предмет удосконалення.

Міні-висновок за розділом. Встановлено, що розробляється — відмовостійкий контентний шар з уніфікованим поданням різноманітного контенту й незалежністю від конкретного джерела метаданих; чому не застосовано готове рішення — наявні системи орієнтовані на один тип контенту й не передбачають підмінності джерел; для чого це потрібно — побудова такого шару є передумовою функціонування інформаційної системи розповсюдження контенту за підпискою. Це обґрунтовує предмет і задачі подальшого проектування.

РОЗДІЛ 2. ПРОЄКТУВАННЯ ТА РЕАЛІЗАЦІЯ ПІДСИСТЕМИ

Розділ присвячено проєктним рішенням та їх реалізації. Викладено порівняльний вибір зовнішніх джерел метаданих, проєкт кешувального агрегатора, інтеграцію джерел, комунікаційні інтерфейси, підсистему персональних колекцій та клієнтську частину. Фрагменти коду винесено у додаток А із посиланнями за текстом.

2.1. Порівняльний вибір зовнішніх джерел метаданих

Вибір джерел виконано методом порівняльного аналізу за критеріями: відсутність обов'язкової авторизації, наявність обкладинок, описів і жанрів, обмеження темпу звернень та стабільність доступу. Результати наведено у таблиці 2.1.

Таблиця 2.1 — Порівняльний аналіз джерел метаданих

Джерело	Без авторизації	Обкладинки	Описи	Жанри	Стабільність доступу
Marvel Developer API	—	+	+	+	—
ComicVine	—	+	+	+	+
TMDB	—	+	+	+	+
Jikan v4 (MyAnimeList)	+	+	+	+	+
akabab/superhero-api	+	+	+	—	+
TVMaze	+	+	+	+	+
Open Library	+	+	+	+	+

За результатами аналізу для аніме й манги обрано Jikan v4, для персонажів коміксів — akabab/superhero-api, для серіалів — TVMaze, для коміксів — Open Library. Джерела, що потребують авторизації або підпису кожного звернення (Marvel, ComicVine, TMDB), відхилено; додатковим підтвердженням ризику залежності від комерційного джерела стало припинення доступу до Marvel Developer API під час виконання роботи, після чого джерело коміксів було замінено на Open Library без зміни клієнтської частини — завдяки відв'язуванню через комунікаційні інтерфейси (п. 2.3).

2.2. Кешувальний агрегатор метаданих

Агрегатор спроектовано за стратегією «кеш-обабіч» (Cache-Aside) над кешем у пам'яті. Для уникнення дублювання коду застосовано узагальнений (generic) метод кешованого посторінкового отримання даних, параметризований сирым типом постачальника та власним типом моделі (приклад 1). Інтервали актуальності диференційовано за характером даних: списки — короткий інтервал, деталі — середній, статичні набори — тривалий.

Зовнішнє джерело Jikan накладає обмеження темпу звернень (близько трьох на секунду). Під час одночасного формування кількох добірок частина звернень відхилялася із кодом «занадто багато запитів», унаслідок чого окремі ряди залишалися порожніми. Для усунення цього синтезовано механізм обмеження темпу звернень: звернення серіалізуються процесним семафором із мінімальним інтервалом між ними та одним повтором після паузи у разі відхилення (приклад 2). Це гарантує дотримання обмежень джерела без втрати даних.

Для виключення контенту для дорослих із загальнодоступних добірок реалізовано фільтр жанрів манги, що доповнює недостатній штатний параметр джерела (приклад 3).

2.3. Інтеграція джерела коміксів (Open Library)

Джерело коміксів інтегровано через комунікаційний інтерфейс `IComicMetadataService`, реалізація якого `OpenLibraryComicService` звертається до відкритого API Open Library. Базову добірку сформовано як об'єднання курованих запитів за супергеройськими франшизами з усуненням повторів та чергуванням джерел (приклад 4); вільнотекстовий пошук обмежено предметною рубрикою коміксів. Сирі відповіді відображаються в уніфіковану модель: побудова адрес обкладинок, виокремлення авторів і року, очищення переліку жанрів (приклади 5, 6). Завдяки реалізації через інтерфейс заміна джерела не торкається ані комунікаційного контролера, ані клієнтської частини.

2.4. Комунікаційні інтерфейси (контролери)

Єдину точку доступу до метаданих реалізовано контролером `MetadataController`, який надає ендпоінти за всіма категоріями контенту — аніме, манга, персонажі, серіали та комікси (приклад 7). Контролер залежить від інтерфейсів сервісів (`IJikanService`, `ISuperheroService`, `IComicMetadataService`), а не від конкретних реалізацій, що відповідає принципу інверсії залежностей. Сервіс персонажів після одноразового завантаження статичного набору виконує фільтрацію та посторінкове гортання повністю у пам'яті (приклад 8).

2.5. Підсистема персональних колекцій

Персональні колекції реалізовано окремою структурною сутністю, відв'язаною від таблиці локальних коміксів, що дозволяє зберігати одиниці контенту з будь-якого зовнішнього джерела. Колекції прив'язано до профілю: активний профіль визначається із заголовка звернення та звіряється з власником облікового запису (приклад 9). Це реалізує вимогу «кожен профіль має власні колекції». Дві колекції — «Бібліотека» (комікси й манга) та «Мій список» (аніме й серіали) — розрізняються дискримінатором, що зменшує кількість структурних сутностей.

2.6. Клієнтська частина

Клієнтську частину побудовано навколо уніфікованого подання контенту. Єдиний компонент картки відображає однаково комікси, мангу, аніме та серіали; за відмови завантаження обкладинки застосовується градієнтний заміник із назвою, що зберігає цілісність сітки (приклад 11). Горизонтальні ряди підтримують дворядкове подання та гортання кнопками (приклад 13). Наскрізний пошук виконує паралельні звернення за кількома типами контенту з усуненням «миготіння» введення (debounce) і відкриває детальне подання безпосередньо в накладенні; для коректного позиціонування накладення винесено у портал кореневого вузла документа (приклад 12). Контекст персональних колекцій перезавантажується при зміні активного профілю, що й забезпечує окремі колекції для кожного профілю (приклад 10). Поточковий перегляд реалізовано вбудованим програвачем стандарту HTML5 без сторонніх залежностей.

РОЗДІЛ 3. ВИПРОБУВАННЯ ТА ОЦІНКА РЕЗУЛЬТАТІВ

Розділ присвячено перевірці підсистеми після першого успішного запуску: відповідності технічному завданню, функціональному випробуванню, перевірці відмовостійкості та обходу обмежень зовнішніх джерел, оцінці продуктивності й засобам контролю якості.

3.1. Відповідність технічному завданню

Перевіркою проходженням сценаріїв підтверджено реалізацію всіх функціональних вимог: єдиної точки доступу до метаданих; уніфікованого каталогу з фільтрацією та гортанням; наскрізного пошуку за кількома типами контенту з відкриттям деталей у накладенні; персональних колекцій на рівні профілю; потокового перегляду відео.

3.2. Функціональне випробування

Перевірено: наповнення всіх рядів каталогу реальними метаданими (обкладинки, описи, жанри); коректне розділення персональних колекцій між профілями (перемикання профілю змінює вміст «Бібліотеки» та «Мого списку»); відкриття деталей результату пошуку в накладенні без переходу на іншу сторінку; виключення контенту для дорослих із загальнодоступних добірок.

3.3. Випробування відмовостійкості та обходу обмежень

Підтверджено усунення проблеми порожніх рядів: після впровадження обмеження темпу звернень добірки, що формуються одночасно, наповнюються повністю, оскільки звернення більше не відхиляються джерелом за перевищення темпу. Перевірено градаційне зниження функціональності: за недоступності зовнішнього джерела підсистема повертає порожню добірку чи резервний набір замість аварійного завершення. Окремо підтверджено сценарій підміни джерела: після припинення доступу до комерційного сервісу коміксів джерело замінено відкритим без зміни клієнтської частини, що засвідчує коректність відв'язування через комунікаційні інтерфейси.

3.4. Оцінка продуктивності

Повторні звернення до каталогу обслуговуються з кешу за час порядку мілісекунд, без звернень до мережі. Статичний набір даних персонажів (близько 1 МБ) завантажується одноразово й кешується на тривалий інтервал, після чого пошук, фільтрація та гортання виконуються у пам'яті. Завдяки мінімальному інтервалу між зверненнями до Jikan дотримується обмеження темпу джерела навіть під час інтенсивного гортання.

3.5. Контроль якості та командна взаємодія

Розроблення велося із застосуванням системи контролю версій Git за моделлю гілок, запитів на злиття та взаємного рецензування коду. Типобезпеку перевіряв компілятор TypeScript, складання — засоби Vite, стиль коду — статичний аналізатор. Інтеграцію з підсистемою облікових записів здійснено через узгоджені комунікаційні інтерфейси та спільний контекст автентифікації.

3.6. Підсумкова оцінка

Функціональність, передбачена технічним завданням, реалізована повністю. Контентний шар забезпечує уніфіковане подання різноманітного контенту, наскрізний пошук, персональні колекції та потоковий перегляд, зберігаючи працездатність за відмови чи обмежень зовнішніх джерел. Обрані рішення підтвердили доцільність; підсистема є придатною до підміни й розширення джерел метаданих без модифікації клієнта.

ВИСНОВКИ

У ході виконання випускної кваліфікаційної роботи спроектовано та впроваджено відмовостійкий контентний шар інформаційної системи Bazinga. Висновки за поставленими у вступі задачами наведено нижче.

- **Задача 1 (огляд і порівняння).** Проведено огляд систем розповсюдження контенту та порівняльний аналіз як споживчих систем (таблиця 1.1), так і доступних джерел метаданих (таблиця 2.1). Встановлено критерії вибору джерел і визначено предмет удосконалення — уніфіковане подання різноманітного контенту з незалежністю від конкретного джерела.
- **Задача 2 (агрегатор).** Спроектовано кешувальний агрегатор за стратегією Cache-Aside з диференційованими інтервалами актуальності та узагальненим методом кешованого гортання, що усунув дублювання коду між однотипними методами.
- **Задача 3 (інтеграція й стійкість).** Інтегровано чотири відкриті джерела метаданих без авторизації; синтезовано механізм обмеження темпу звернень, що усунув відхилення звернень за перевищення лімітів і пов'язану з ним проблему порожніх рядів; забезпечено градаційне зниження функціональності за відмови джерел.
- **Задача 4 (каталог, пошук, відео).** Реалізовано уніфікований каталог зі спільним компонентом подання різноманітного контенту, наскрізний пошук за чотирма типами контенту з відкриттям деталей у накладенні та потоковий перегляд відео вбудованим програвачем HTML5.
- **Задача 5 (персональні колекції).** Реалізовано підсистему персональних колекцій («Бібліотека» та «Мій список») з прив'язкою до профілю; підтверджено, що кожен профіль має власні колекції.
- **Задача 6 (випробування).** Проведено функціональне випробування й перевірку відмовостійкості; підтверджено наповнення рядів після впровадження обмеження темпу звернень, обслуговування повторних звернень з кешу за час порядку мілісекунд та коректність підміни джерела без зміни клієнта.

Наукова новизна роботи полягає у синтезі уніфікованої інформаційної моделі подання різноманітного контенту на рівні програмного інтерфейсу та у застосуванні кешування з обмеженням темпу звернень і відв'язуванням від постачальника метаданих, що забезпечує відмовостійкість в умовах неповної визначеності. Практична цінність визначається застосовністю результатів у системах розповсюдження контенту та агрегаторах метаданих.

Перспективними напрямками подальших досліджень є впровадження адаптивного потокового відео (HLS/DASH), застосування мережі доставки контенту (CDN) для статичних ресурсів та побудова рекомендаційної підсистеми на основі колаборативної фільтрації.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. ДСТУ 3008:2015. Інформація та документація. Звіти у сфері науки і техніки. Структура та правила оформлювання. Київ: ДП «УкрНДНЦ», 2016.
 2. ДСТУ 8302:2015. Інформація та документація. Бібліографічне посилання. Загальні положення та правила складання. Київ: ДП «УкрНДНЦ», 2016.
 3. Величко О. М., Гордієнко Т. Б. Інтелектуальні інформаційні системи: структура і застосування: підручник. Херсон, 2022. 728 с. ISBN 978-966-289-552-0.
 4. Анісімов А. В., Кулябко П. П. Інформаційні системи та бази даних: навчальний посібник. Київ, 2021. 110 с.
 5. R. Fielding, J. Reschke. Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content. RFC 7231. IETF, 2014. URL: <https://datatracker.ietf.org/doc/html/rfc7231>
 6. Cache-Aside pattern. Microsoft Azure Architecture Center, 2024. URL: <https://learn.microsoft.com/azure/architecture/patterns/cache-aside>
 7. ASP.NET Core documentation. Microsoft, 2024. URL: <https://learn.microsoft.com/aspnet/core>
 8. Caching in .NET (IMemoryCache). Microsoft, 2024. URL: <https://learn.microsoft.com/aspnet/core/performance/caching/memory>
 9. React documentation. Meta Open Source, 2024. URL: <https://react.dev/learn>
 10. TypeScript documentation. Microsoft, 2024. URL: <https://www.typescriptlang.org/docs/>
 11. Vite documentation. 2024. URL: <https://vite.dev/guide/>
 12. TanStack Query documentation. 2024. URL: <https://tanstack.com/query/latest>
 13. HTML Living Standard: The video element. WHATWG, 2024. URL: <https://html.spec.whatwg.org/multipage/media.html>
 14. Jikan REST API (v4) documentation. 2024. URL: <https://docs.api.jikan.moe/>
 15. TVMaze API documentation. 2024. URL: <https://www.tvmaze.com/api>
 16. Open Library Developer APIs. Internet Archive, 2024. URL: <https://openlibrary.org/developers/api>
 17. akabab/superhero-api. 2024. URL: <https://akabab.github.io/superhero-api/>
 18. MySQL 8.0 Reference Manual. Oracle, 2024. URL: <https://dev.mysql.com/doc/refman/8.0/en/>
 19. Репозиторій проєкту Bazinga. URL: <https://github.com/dentuss/Bazinga>
-

ДОДАТОК А. ПРИКЛАДИ ПРОГРАМНОГО КОДУ

У додатку наведено приклади програмного коду, що ілюструють ключові проєктні рішення контентної підсистеми. Повний вихідний код розміщено у репозиторії проєкту (джерело 19 переліку використаних джерел).

Приклад 1 — Узагальнений метод кешованого посторінкового отримання даних за стратегією Cache-Aside (`Services/JikanService.cs`)

```
private async Task<JikanPagedResponse<TDto>> GetPagedCached<TRaw, TDto>(
    string path, TimeSpan ttl, Func<TRaw, TDto> map, CancellationToken ct)
    where TRaw : class
{
    return (await _cache.GetOrCreateAsync($"path:{path}", async entry =>
    {
        entry.AbsoluteExpirationRelativeToNow = ttl;
        var raw = await GetJsonAsync<JikanEnvelope<List<TRaw>>>(path, ct);
        return new JikanPagedResponse<TDto>
        {
            Data = raw?.Data?.Select(map).ToList() ?? new(),
            Pagination = new JikanPagination
            {
                CurrentPage = raw?.Pagination?.CurrentPage ?? 1,
                LastVisiblePage = raw?.Pagination?.LastVisiblePage ?? 1,
                HasNextPage = raw?.Pagination?.HasNextPage ?? false,
                TotalItems = raw?.Pagination?.Items?.Total,
            },
        };
    })))!;
}
```

Приклад 2 — Обмеження темпу звернень до зовнішнього джерела з одним повтором (`Services/JikanService.cs`)

```
// Сериалізуємо звернення процесним семафором із мінімальним інтервалом, щоб
// не перевищувати ~3 запити/с і не отримувати відмов «занадто багато запитів».
private static readonly SemaphoreSlim Gate = new(1, 1);
private static readonly TimeSpan MinSpacing = TimeSpan.FromMilliseconds(380);
private static DateTime _lastCallUtc = DateTime.MinValue;

private async Task<HttpResponseMessage> SendThrottledAsync(string path, CancellationToken
ct)
{
    await Gate.WaitAsync(ct);
    try
    {
        var since = DateTime.UtcNow - _lastCallUtc;
        if (since < MinSpacing) await Task.Delay(MinSpacing - since, ct);
        try { return await _http.GetAsync(path, ct); }
        finally { _lastCallUtc = DateTime.UtcNow; }
    }
    finally { Gate.Release(); }
}

private async Task<T?> GetJsonAsync<T>(string path, CancellationToken ct)
{
    for (var attempt = 0; attempt < 2; attempt++)
    {
        using var resp = await SendThrottledAsync(path, ct);
        if (resp.StatusCode == System.Net.HttpStatusCode.TooManyRequests)
        {

```

```

        if (attempt == 0) { await Task.Delay(TimeSpan.FromMilliseconds(900), ct);
continue; }
        return default;
    }
    if (!resp.IsSuccessStatusCode) return default;
    return await resp.Content.ReadFromJsonAsync<T>(Json, ct);
}
return default;
}

```

Приклад 3 — Виключення жанрів для дорослих із добірок манги (Services/JikanService.cs)

```

// 9 = Ecchi, 12 = Hentai, 28 = Boys Love, 30 = Girls Love, 49 = Erotica.
// Штатний параметр sfw=true фільтрує лише Hentai, тож доповнюємо явним списком.
private const string MangaExcludedGenres = "9,12,28,30,49";

public Task<JikanPagedResponse<MangaDto>> SearchMangaAsync(
    string? query, int page, int limit, int? genre, string? status, string? orderBy,
    Cancellation token ct)
{
    var url = BuildUrl("manga", q =>
    {
        q["page"] = page.ToString(); q["limit"] = limit.ToString();
        if (!string.IsNullOrEmpty(query)) q["q"] = query;
        q["sfw"] = "true";
        q["genres_exclude"] = MangaExcludedGenres;
    });
    return GetPagedCached<JikanManga, MangaDto>(url, ListCacheTtl, MapManga, ct);
}

```

Приклад 4 — Куроване об'єднання запитів з усуненням повторів і чергуванням франшиз (Services/OpenLibraryComicService.cs)

```

private async Task<List<ComicMetaDto>> LoadCuratedAsync(Cancellation token ct)
{
    return (await _cache.GetOrCreateAsync("comics:ol:curated", async entry =>
    {
        entry.AbsoluteExpirationRelativeToNow = CuratedTtl;
        var tasks = CuratedFranchises.Select(f => SearchRawAsync(f, 1, 18, ct));
        var results = await Task.WhenAll(tasks);

        // Чергуємо франшизи (round-robin), усуваємо повтори за id, вимагаємо обкладинку.
        var byFranchise = results.Select(r => r.Data).ToList();
        var seen = new HashSet<int>();
        var merged = new List<ComicMetaDto>();
        var max = byFranchise.Count == 0 ? 0 : byFranchise.Max(l => l.Count);
        for (var i = 0; i < max; i++)
            foreach (var list in byFranchise)
            {
                if (i >= list.Count) continue;
                var item = list[i];
                if (item.Image is null) continue;
                if (seen.Add(item.Id)) merged.Add(item);
            }
        return merged;
    }));
}

```

Приклад 5 — Відображення сирової відповіді у модель та побудова адрес обкладинок (Services/OpenLibraryComicService.cs)

```
private static ComicMetaDto MapDoc(OldDoc d) => new()
{
    Id = ParseWorkId(d.Key), // "/works/OL123W" → 123
    Title = d.Title ?? "Untitled",
    Image = CoverUrl(d.CoverId, "L"),
    Thumbnail = CoverUrl(d.CoverId, "M"),
    Year = d.FirstPublishYear,
    Creators = (d.AuthorName ?? new()).Take(4).ToList(),
    Genres = CleanSubjects(d.Subject),
};

private static string? CoverUrl(int? coverId, string size) =>
    coverId is > 0 ? $"https://covers.openlibrary.org/b/id/{coverId}-{size}.jpg" : null;
```

Приклад 6 — Очищення переліку жанрів від службових позначок (Services/OpenLibraryComicService.cs)

```
private static List<string> CleanSubjects(IEnumerable<string>? subjects) =>
    (subjects ?? Enumerable.Empty<string>())
        .Where(s => !string.IsNullOrEmpty(s)
            && !s.Contains(':') && !s.Contains('=') && s.Length <= 28
            && !s.StartsWith("Reading Level", StringComparison.OrdinalIgnoreCase)
            && !s.StartsWith("Accessible book", StringComparison.OrdinalIgnoreCase))
        .Distinct(StringComparer.OrdinalIgnoreCase)
        .Take(6)
        .ToList();
```

Приклад 7 — Єдина точка доступу до метаданих з інверсією залежностей (Controllers/MetadataController.cs)

```
public MetadataController(IJikanService jikan, ISuperheroService supers,
    IComicMetadataService comics)
{
    _jikan = jikan; _supers = supers; _comics = comics; // залежність від інтерфейсів
}

[HttpGet("comics")]
public Task<ComicsMetaPagedResponse> Comics(
    [FromQuery] int page = 1, [FromQuery] int limit = 24, [FromQuery] string? q = null,
    CancellationToken ct = default)
    => _comics.ListAsync(Math.Max(1, page), Math.Clamp(limit, 1, MaxComicsLimit), q, ct);

[HttpGet("anime/top")]
public Task<JikanPagedResponse<AnimeDto>> TopAnime(
    [FromQuery] int page = 1, [FromQuery] int limit = 12, CancellationToken ct = default)
    => _jikan.TopAnimeAsync(Math.Max(1, page), Clamp(limit), ct);
```

Приклад 8 — Фільтрація та посторінкове гортання статичного набору у пам'яті (Services/SuperheroService.cs)

```
public async Task<JikanPagedResponse<SuperheroDto>> ListAsync(
    int page, int limit, string? query, string? publisher, string? alignment,
    CancellationToken ct)
{
    var all = await LoadAllAsync(ct); // одноразове завантаження + кеш на 12
    год
    IEnumerable<SuperheroDto> filtered = all;
    if (!string.IsNullOrEmpty(publisher))
        filtered = filtered.Where(h => string.Equals(h.Publisher, publisher,
            StringComparison.OrdinalIgnoreCase));

    var list = filtered.ToList();
    var safePage = Math.Max(1, page);
```

```

var safeLimit = Math.Clamp(limit, 1, 48);
var pageItems = list.Skip((safePage - 1) * safeLimit).Take(safeLimit).ToList();
return new JikanPagedResponse<SuperheroDto>
{
    Data = pageItems,
    Pagination = new JikanPagination
    {
        CurrentPage = safePage,
        LastVisiblePage = Math.Max(1, (int)Math.Ceiling(list.Count /
(double)safeLimit)),
        HasNextPage = safePage * safeLimit < list.Count,
        TotalItems = list.Count,
    },
};
}

```

Приклад 9 — Додавання до колекції з прив'язкою до профілю та звіркою власника (`Controllers/CollectionsController.cs`)

```

[HttpPost("{collection}")]
public async Task<ActionResult<IEnumerable<CollectionItemDto>>> Add(
    string collection, [FromBody] CollectionItemRequest req, CancellationToken ct)
{
    collection = collection.ToLowerInvariant();
    if (!AllowedCollections.Contains(collection)) return NotFound();
    var profile = await ResolveProfileAsync(ct); // профіль із заголовка, звірений із
    власником
    if (profile is null) return Unauthorized();

    var exists = await _db.CollectionItems.AnyAsync(
        i => i.ProfileId == profile.Id && i.Collection == collection
        && i.Kind == req.Kind && i.ContentId == req.ContentId, ct);
    if (!exists)
    {
        _db.CollectionItems.Add(new ProfileCollectionItem {
            ProfileId = profile.Id, Collection = collection, Kind =
req.Kind.ToLowerInvariant(),
            ContentId = req.ContentId.Trim(), Title = req.Title, Image = req.Image,
            Subtitle = req.Subtitle, PayloadJson = req.PayloadJson, AddedAt =
DateTime.UtcNow });
        try { await _db.SaveChangesAsync(ct); }
        catch (DbUpdateException) { /* гонитва за унікальним індексом – ігноруємо */ }
    }
    return Ok(await LoadItemsAsync(profile.Id, collection, null, ct));
}

```

Приклад 10 — Перезавантаження персональних колекцій при зміні активного профілю (`contexts/CollectionsContext.tsx`)

```

const reload = useCallback(async () => {
    if (!token || profileId == null) { setLibrary([]); setMyList([]); return; }
    const [lib, list] = await Promise.all([
        listCollection(token, profileId, "library"),
        listCollection(token, profileId, "mylist"),
    ]);
    setLibrary(lib); setMyList(list);
}, [token, profileId]);

// Перезавантаження при зміні акаунта чи активного профілю – саме це й забезпечує
// окремі колекції для кожного профілю.
useEffect(() => { void reload(); }, [reload]);

```

Приклад 11 — Уніфікована картка з градієнтним заміником при відмові завантаження обкладинки (`components/MediaCard.tsx`)

```
<div className="relative aspect-[2/3] overflow-hidden">
  <div aria-hidden className={cn("absolute inset-0 bg-gradient-to-br", fallbackBg)} />
  <div aria-hidden className="absolute inset-0 grid place-items-center text-center px-3">
    <span className="text-xs font-black uppercase tracking-wider text-white/85 line-clamp-3">
      {title}
    </span>
  </div>
  <img
    src={resolveImageUrl(image)} alt={title} loading="lazy"
    className="relative h-full w-full object-cover transition-transform duration-500 group-hover:scale-110"
    onError={(e) => ((e.currentTarget as HTMLImageElement).style.opacity = "0")}
  />
</div>
```

Приклад 12 — Наскрізний пошук: паралельні звернення та винесення накладення у портал (`components/MasterSearch.tsx`)

```
const heroes = useQuery({ queryKey: ["ms:characters", debounced],
  queryFn: () => fetchSuperheroes({ q: debounced, limit: 5 }), enabled, staleTime: 60_000 });
const manga = useQuery({ queryKey: ["ms:manga", debounced],
  queryFn: () => searchManga({ q: debounced, limit: 5 }), enabled, staleTime: 60_000 });
const anime = useQuery({ queryKey: ["ms:anime", debounced],
  queryFn: () => searchAnime({ q: debounced, limit: 5 }), enabled, staleTime: 60_000 });
const metaIssues = useQuery({ queryKey: ["ms:comics", debounced],
  queryFn: () => fetchComicsMeta({ q: debounced, limit: 5 }), enabled, staleTime: 60_000 });

// Накладення виноситься у портал кореня документа, щоб його не «обрізав» хедер.
return createPortal(overlay, document.body);
```

Приклад 13 — Дворядкове горизонтальне подання ряду каталогу (`components/Rail.tsx`)

```
<div
  id={trackId(id)}
  className={cn(
    "overflow-x-auto px-4 md:px-8 pb-2 pt-6 scroll-smooth no-scrollbar",
    rows === 2
      ? "grid grid-rows-2 grid-flow-col auto-cols-max gap-x-4 gap-y-5 md:gap-x-5"
      : "flex gap-4 md:gap-5 snap-x snap-mandatory"
  )}
  style={{ scrollbarWidth: "none" }}
>
  {children}
</div>
```